

CS-1204
Object-Oriented Programming
(3+1)

Week 05

By

Engr. Muhammad Awais

Contents

- Composition
- Aggregation

Aggregation

- Aggregation is a **has-a relationship** where one class contains another class, but the contained object can exist independently of the container.
- Lifespan: The **contained object is not destroyed** when the container is destroyed.
- Weak Relationship
- Whole-part relationship
- Loose Coupling
- Team ◇— Player

Aggregation Example

```
#include <iostream>
#include <string>
using namespace std;

class Player {
    string name;
public:
    Player(string n) {
        name = n;
    }

    void play() {
        cout << name << " is playing.\n";
    }

    ~Player() {
        cout << name << " is leaving the field.\n";
    }
};
```

```
class Team {
    Player* player; // Aggregation: Team has a Player, but doesn't own it
public:
    Team(Player* p) {
        player = p;
    }

    void startMatch() {
        cout << "Match started.\n";
        player->play();
    }
};
```

// No destructor needed to delete player - not owned!

```
int main() {
    Player* p1 = new Player("Alice"); // Player created independently
    {
        Team t1(p1); // Team aggregates existing player
        t1.startMatch();
        // Team goes out of scope and is destroyed
        // But Player is still alive
    }

    p1->play(); // Still accessible after Team is gone
    delete p1; // Now we manually destroy Player

    return 0;
}
```

Key Points

- Team does not delete Player.
- The player is created outside the Car. It exists independently.
- Player can exist before and after the Team object.
- Shows weak ownership and independent lifetimes—hallmark of Aggregation.

Composition

- Composition is a **strong has-a relationship** where the container class owns the contained object.
- Lifespan: The **contained object is destroyed** when the container is destroyed.
- Whole-part relationship
- Strong Relationship
- Tightly Coupled
- Car ◆—— Engine

Composition Example

```
class Engine {
    string engineNumber;
public:
    Engine() {
        engineNumber = "UNKNOWN";
        cout << "Default Engine created.\n";
    }

    void setEngineNumber(string number) {
        engineNumber = number;
    }

    void start() {
        cout << "Engine " << engineNumber << " starts.\n";
    }

    ~Engine() {
        cout << "Engine " << engineNumber << " destroyed.\n";
    }
};
```

```
class Car {
    Engine engine; // Composition: Car owns Engine
public:
    Car(string engineNum) {
        engine.setEngineNumber(engineNum); // Initialized inside constructor body
        cout << "Car created with engine " << engineNum << ".\n";
    }

    void startCar() {
        engine.start();
        cout << "Car is starting.\n";
    }

    ~Car() {
        cout << "Car destroyed.\n";
    }
};
```

```
int main() {
    {
        Car myCar("ENG-9876"); // Car and Engine constructed together
        myCar.startCar();
        // When Car goes out of scope, Engine is destroyed too
    }

    return 0;
}
```

Key Points

- Car and engine has strong relationship.
- Car class owns the engine.
- Engine can not exist independently.
- Engine object is destroyed when the car is destroyed.

Example code with Virtual Inheritance

```
#include <iostream>
using namespace std;

class Person {
public:
    void showIdentity() {
        cout << "I am a person." << endl;
    }
};

// Use virtual inheritance
class Teacher : virtual public Person {
public:
    void teach() {
        cout << "Teaching students." << endl;
    }
};

class Student : virtual public Person {
public:
    void study() {
        cout << "Studying courses." << endl;
    }
};

class TeachingAssistant : public Teacher, public Student {
public:
    void assist() {
        cout << "Assisting in labs and grading." << endl;
    }
};
```

```
int main() {
    TeachingAssistant ta;
    ta.showIdentity(); //  No ambiguity
    ta.teach();
    ta.study();
    ta.assist();
    return 0;
}
```

Comparison

Feature	Aggregation	Composition
Ownership	No (shared object)	Yes (exclusive ownership)
Lifetime	Independent	Dependent
Deletion impact	Deleting container has no effect	Deleting container destroys parts
Reusability	Higher (same object can be reused)	Lower (tightly coupled)
Implementation	Pointer or reference	Direct member (object)